

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 АНАЛИТИЧЕСКИЙ РАЗДЕЛ.....	5
1.1 Анализ рынка (Обзор конкурентов и существующих решений)	5
1.1.1 Конкурент 1 – Яндекс Карты / 2ГИС	5
1.1.2 Конкурент 2 – TripAdvisor.....	6
1.1.3 Конкурент 3 – Restoclub	6
1.1.4 Сравнительный анализ.....	7
1.2 Анализ целевой аудитории.....	7
1.2.1 Описание целевой аудитории	8
1.2.2 Проблемы целевой аудитории	8
2 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ	9
2.1 Характеристика веб-приложения	9
2.2 Требования к технологическому решению	10
2.3 Архитектура приложения	11
3 ПРАКТИЧЕСКИЙ РАЗДЕЛ.....	13
3.1 Фронтенд-разработка	13
3.2 Бэкенд-разработка	14
3.3 Визуализация интерфейса	15
3.4 Тестирование, оценка и рекомендации	18
ЗАКЛЮЧЕНИЕ	21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	22

ВВЕДЕНИЕ

В настоящее время выбор места для досуга требует учёта множества факторов: атмосферы, типа заведения, ценового сегмента, личных предпочтений. Существующие агрегаторы (например, «Афиша», TripAdvisor, Яндекс Карты) предлагают общий рейтинг, но не учитывают динамическую историю посещений и не дают персонализированных рекомендаций с возможностью настройки детальных фильтров.

В данной работе будет рассмотрен процесс создания веб-приложения «Vitera», которое помогает пользователям находить подходящие места для отдыха на основе их предпочтений (атмосфера, тип места, дополнительные опции, цена) и анализирует уже посещённые локации для формирования рекомендаций.

Актуальность предлагаемого решения заключается в отсутствии на рынке простых и интуитивных инструментов для персонализированного подбора мест досуга с учётом нескольких критериев одновременно, а также с возможностью ведения личной истории посещений без обязательной регистрации.

Целью работы является разработка веб-приложения «Vitera» для автоматического подбора мест отдыха и развлечений в крупных городах России.

Для выполнения поставленной цели будут решены следующие задачи:

- Проведение анализа конкурентных решений.
- Определение проблем, существующих на рынке и проблем целевой аудитории.
- Разработка веб-приложения (фронтенд и бэкенд) с функциями фильтрации, рекомендаций и истории посещений.

1 АНАЛИТИЧЕСКИЙ РАЗДЕЛ

В данном разделе происходит анализ рынка (конкурентных решений), анализ целевой аудитории и ее проблем, а также описываются требования к разработке.

1.1 Анализ рынка (Обзор конкурентов и существующих решений)

В данном подразделе будут рассмотрены сайты и приложения-конкуренты, предлагающие схожие решения, а также выявлены их сильные и слабые стороны.

На сегодняшний день существует несколько популярных сервисов, позволяющих пользователям находить рестораны, парки, музеи, бары и другие места для отдыха. Однако большинство из них либо не предоставляют персонализированных рекомендаций, либо имеют перегруженный интерфейс, либо не учитывают историю посещений пользователя.

В рамках анализа были выбраны три основных конкурента, наиболее близких по функциональности к разрабатываемому приложению «Vitera»:

1.1.1 Конкурент 1 – Яндекс Карты / 2ГИС

Прямым конкурентом разрабатываемого продукта являются картографические сервисы с функциями поиска организаций (Яндекс Карты, 2ГИС). Они предоставляют обширную базу мест, отзывы, рейтинги и информацию о часах работы.

Сильные стороны: огромная база, интеграция с навигацией, актуальные отзывы.

Слабые стороны: отсутствие персонализированных рекомендаций на основе истории посещений пользователя; невозможность тонкой настройки по

атмосфере или типу кухни; интерфейс ориентирован на поиск адреса, а не на подбор впечатлений.

1.1.2 Конкурент 2 – TripAdvisor

При рассмотрении существующих решений стоит отметить международный сервис TripAdvisor, который специализируется на отзывах о местах отдыха, отелях, ресторанах и достопримечательностях.

Сильные стороны: огромное количество отзывов и оценок, удобная система рейтинга.

Слабые стороны: интерфейс перегружен рекламой и второстепенной информацией; персонализация ограничена ручным поиском; нет функции ведения личной истории посещений с автоматическим формированием новых рекомендаций.

1.1.3 Конкурент 3 – Restoclub

Третьим конкурентом является российский сервис Restoclub, ориентированный на поиск и бронирование ресторанов, кафе и баров. Сервис предоставляет подробные карточки заведений с меню, акциями и возможностью онлайн-бронирования столиков.

Сильные стороны: удобный поиск по кухне, среднему чеку и району; наличие актуальных акций и скидок; интеграция с системой бронирования.

Слабые стороны: функционал ограничен преимущественно ресторанами и кафе (отсутствуют парки, музеи, спортивные объекты); настройка по атмосфере (романтическая, деловая и т.д.) не предусмотрена; нет возможности сохранять историю посещений и получать персонализированные рекомендации на основе предпочтений пользователя.

1.1.4 Сравнительный анализ

Для наглядного сравнения разрабатываемого приложения «Vitera» с существующими конкурентами была составлена таблица 1.1, в которой оценивается наличие ключевых функций.

Таблица 1.1 – Сравнение функциональных возможностей

Критерий	Яндекс Карты / 2ГИС	TripAdvisor	Restoclub	Vitera
Настройка по атмосфере	—	—	—	+
Фильтрация по типу места (парки, музеи и т.д.)	+/-	+	—	+
Детальные подкатегории (кухня, вид парка)	—	+/-	+	+
Учёт истории посещений	—	—	—	+
Персональные рекомендации	—	—	—	+
Процент совпадения с предпочтениями	—	—	—	+
Отметка посещённых мест	—	—	—	+
Смена города с сохранением данных	+	+	+	+

Примечание: «+» – функция присутствует, «–» – отсутствует, «+/-» – частично.

Таким образом, по большинству критериев персонализации и ведения истории разрабатываемое приложение «Vitera» превосходит существующие аналоги, что подтверждает его актуальность и востребованность.

1.2 Анализ целевой аудитории

В данном разделе будет описана целевая аудитория проекта «Vitera», её потребности, проблемы и ожидания от веб-приложения.

1.2.1 Описание целевой аудитории

Целевой аудиторией проекта являются люди среднего дохода, в основном жители крупных городов России (Москва, Санкт-Петербург, Казань, Екатеринбург, Новосибирск, Нижний Новгород, Челябинск, Самара, Омск, Ростов-на-Дону, Уфа, Красноярск) в возрасте от 18 до 45 лет. Это активные люди, которые регулярно посещают рестораны, бары, парки, музеи, кинотеатры, ищут новые места для свиданий, встреч с друзьями, семейного отдыха или уединённых прогулок. Аудитория имеет базовые навыки работы с веб-приложениями, ценит скорость работы сервиса, простоту интерфейса и возможность персонализации.

1.2.2 Проблемы целевой аудитории

34 % опрошенных (гипотетические данные на основе анализа рынка) отмечают, что существующие сервисы предлагают слишком много однотипных вариантов, из-за чего сложно сделать выбор.

52 % опрошенных тратят от 15 до 30 минут на поиск подходящего места, поскольку агрегаторы не учитывают личные предпочтения по атмосфере и типу заведения.

41 % опрошенных хотели бы получать рекомендации, основанные на ранее посещённых местах, но такая функция отсутствует в большинстве приложений.

Решением проблемы может послужить создание веб-приложения, которое:

- Хранит историю посещений в локальном хранилище и использует её для формирования рекомендаций.
- Позволяет тонко настраивать предпочтения (атмосфера, тип места, субкатегория, цена) и пересчитывать совпадение в реальном времени.
- Поддерживает переключение между городами, сохраняя предпочтения и историю отдельно для каждого города.

2 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

В данном разделе будут определены основные требования к разрабатываемому веб-приложению.

Требования строятся, основываясь на анализе потребностей целевой аудитории и общих целей проекта. Общими требованиями являются: удобство использования, производительность, интуитивность и понятность для пользователя. Важные аспекты включают функциональность, надежность, безопасность и масштабируемость.

2.1 Характеристика веб-приложения

Основным объектом предметной области является веб-приложение для персонализированного поиска мест досуга «Vitera».

Сфера применения разрабатываемого интерфейса – область организации досуга и развлечений: подбор ресторанов, баров, парков, музеев, кинотеатров и других объектов в крупных городах России.

Назначение разрабатываемого интерфейса – предоставить пользователю удобный инструмент для фильтрации большого массива данных (более 360 мест в 12 городах) и автоматического формирования рейтинга мест по степени совпадения с выбранными параметрами.

Основной функционал заключается в следующем:

- настройка предпочтений (атмосфера, тип места, дополнительные опции, ценовой сегмент);
- получение персональных рекомендаций с процентом совпадения;
- просмотр всех мест выбранного города с возможностью отметки посещений;
- ведение истории посещений (хранится в localStorage);
- сброс истории и сброс предпочтений;
- переключение между городами.

2.2 Требования к технологическому решению

Требования к технологическому решению включают в себя выбор подхода к разработке, программных средств, платформ, фреймворков и языков программирования, которые обеспечат надежность, масштабируемость и производительность разрабатываемого веб-приложения.

Для настоящей работы в качестве программных средств будут использованы: среда разработки Visual Studio Code, система контроля версий Git, фреймворк ASP.NET Core 10, платформа .NET 10.

Для реализации фронтенда использованы:

- HTML5 – для разметки страниц;
- CSS3 – для стилизации и адаптивного дизайна;
- JavaScript (ES6+) – для интерактивности, асинхронных запросов, хранения данных в localStorage и динамической перерисовки интерфейса;
- Bootstrap 5 – для адаптивной вёрстки и готовых компонентов (карточки, модальные окна, навигация);
- Font Awesome – для векторных иконок.

Для реализации бэкенда использованы:

- ASP.NET Core 10 (C#) – для создания REST API и раздачи статических файлов;
- встроенный десериализатор JSON (System.Text.Json) с настройкой регистронезависимости;
- Dependency Injection – для управления сервисами;
- Middleware – для логирования запросов и установки заголовков безопасности (CSP, X-Frame-Options, X-Content-Type-Options);
- статический JSON-файл (places.json) в качестве хранилища данных, загружаемый в память при старте приложения.

Хранилище данных – файл Data/places.json, содержащий более 360 записей о местах в 12 городах. Данные десериализуются в список объектов Place один раз при запуске приложения и хранятся в памяти для быстрого доступа.

Структура JSON-файла

Файл places.json представляет собой массив объектов, каждый из которых соответствует одному месту. Ниже приведён пример одной записи (для Москвы, Кремль):

Листинг 2.1 – Пример структуры JSON-объекта в файле places.json

```
{
  "id": "msk_1",
  "name": "Московский Кремль и Красная площадь",
  "city": "Москва",
  "address": "Красная площадь",
  "atmosphere": "престижная",
  "type": "культура",
  "subcat": "исторические",
  "price": "бюджетно (до 1000₽)",
  "rating": 4.9,
  "description": "Сердце России, главная достопримечательность, объект ЮНЕСКО.",
  "website": "https://www.kreml.ru/"
}
```

Всего в файле содержится более 360 таких записей для 12 городов. Благодаря такому формату обеспечивается лёгкость расширения базы – достаточно добавить новый объект в массив, и после перезапуска приложения данные станут доступны. Для десериализации используется System.Text.Json с настройкой PropertyNameCaseInsensitive = true, что позволяет игнорировать регистр имён полей (например, поле id в JSON сопоставляется со свойством Id в модели C#).

2.3 Архитектура приложения

Разработанное веб-приложение построено по клиент-серверной архитектуре с использованием подхода Single Page Application (SPA) на фронтенде и REST API на бэкенде.

Компоненты системы:

1. Клиентская часть (фронтенд) – HTML, CSS, JavaScript.

Отвечает за отображение интерфейса, взаимодействие с пользователем, отправку HTTP-запросов к API, а также локальное хранение предпочтений и истории посещений в localStorage.

2. Серверная часть (бэкенд) – ASP.NET Core 10.

Обрабатывает запросы клиента, читает JSON-файл с данными о местах, предоставляет эндпоинты для получения списка мест и информации по конкретному id. Статические файлы (фронтенд) раздаются из папки wwwroot.

3. Хранилище данных – JSON-файл.

Выбран для простоты реализации, поскольку объем данных невелик (360 записей), а изменения базы требуются редко.

Поток данных в приложении:

1. Пользователь открывает index.html → браузер загружает статику (HTML, CSS, JS).

2. JavaScript инициирует fetch(/api/places) → бэкенд читает places.json и возвращает JSON.

3. Пользователь настраивает предпочтения → они сохраняются в localStorage.

4. JavaScript ранжирует места на основе предпочтений и отображает карточки.

5. При отметке посещения данные добавляются в localStorage (история).

6. При переключении города история и предпочтения загружаются для нового города из localStorage.

Такая архитектура позволяет легко масштабировать приложение (заменить JSON на базу данных, добавить аутентификацию) и развивать его без полной переписывания логики.

3 ПРАКТИЧЕСКИЙ РАЗДЕЛ

В данной главе будет описан процесс разработки технологического решения и приведены результаты тестирования разработки. Также будут определены рекомендации относительно дальнейшего использования предлагаемого решения.

3.1 Фронтенд-разработка

Для реализации фронтенда веб-приложения были использованы следующие языки программирования: HTML, CSS, JavaScript. Ключевые особенности реализации:

- **Одностраничная архитектура (SPA):** динамическая перерисовка содержимого (рекомендации, все места, история) без перезагрузки страницы. Переключение между вкладками осуществляется через JavaScript, который изменяет содержимое контейнера `#appContainer`.
- **Хранение данных на клиенте:** предпочтения пользователя (атмосфера, тип места, дополнительные опции, цена) и история посещений сохраняются в `localStorage`. Данные привязаны к выбранному городу, что позволяет при смене города загружать соответствующие предпочтения и историю.
- **Модальное окно настроек:** реализовано с помощью `Bootstrap Modal`. При выборе типа места (например, «рестораны») динамически подгружаются дополнительные опции (кухня: итальянская, японская, грузинская и т.д.), что достигается через JavaScript и предварительно заданный объект `extraOptionsMap`.
- **Алгоритм ранжирования мест:** для каждого места вычисляется процент совпадения с предпочтениями пользователя. Весовые коэффициенты: атмосфера – 30%, тип места – 30%, субкатегория – 30%, цена – 10%. Итоговая оценка для сортировки рассчитывается

как $\text{matchPercent} * 0.8 + \text{rating} * 2$, что позволяет учитывать как точность совпадения, так и популярность места.

- **Асинхронная загрузка данных:** при загрузке страницы и при смене города выполняется асинхронный запрос `fetch('/api/places')` для получения списка всех мест с бэкенда. Данные кэшируются в переменной `allPlaces`.
- **Адаптивный дизайн:** благодаря Bootstrap 5 интерфейс корректно отображается на мобильных устройствах, планшетах и десктопах. Использованы CSS Grid и Flexbox для расположения карточек мест.

3.2 Бэкенд-разработка

Для реализации бэкенда рассматриваемого в данной работе веб-приложения были использованы: ASP.NET Core 10, язык C#. Основные компоненты:

- **REST API эндпоинты:**

- GET `/api/places` – возвращает полный список всех мест в формате JSON. Доступен без авторизации.
- GET `/api/places/{id}` – возвращает конкретное место по строковому идентификатору (например, `msk_1`). При отсутствии места возвращает статус 404 с сообщением.
- POST `/api/recommendations` – принимает массив строк `visitedIds` и возвращает рекомендации на основе анализа предпочтений (резервный эндпоинт для будущих расширений).

- **Сервис `PlaceRepository`:**

- Загружает JSON-файл из папки `Data` при инициализации.
- Использует `JsonSerializerOptions` с параметром `PropertyNameCaseInsensitive = true`, что позволяет сопоставлять поля JSON, написанные в нижнем регистре

- (id, name), со свойствами модели C#, написанными с заглавной буквы (Id, Name).
- Предоставляет методы GetAllPlaces() и GetPlaceById(string id).
- Зарегистрирован как Singleton в контейнере DI.
- **Сервис RecommendationService:**
 - Анализирует частоту посещённых категорий и рекомендует места из наиболее популярных категорий, которые пользователь ещё не посещал.
 - Если рекомендаций недостаточно, добавляет места из других категорий с высоким рейтингом.
- **Middleware:**
 - RequestLoggingMiddleware – логирует каждый входящий запрос и статус ответа.
 - SecurityHeadersMiddleware – добавляет заголовки безопасности: X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Content-Security-Policy (ограничение источников скриптов и стилей), Referrer-Policy, Permissions-Policy.
- **Статические файлы:** все HTML, CSS, JS файлы размещены в папке wwwroot и раздаются с помощью UseDefaultFiles() и UseStaticFiles().
- **Обработка ошибок:** реализован fallback на index.html для всех необработанных маршрутов (для поддержки SPA). Для несуществующих API-эндпоинтов возвращается JSON-ошибка.

3.3 Визуализация интерфейса

Ниже представлены основные экраны разработанного веб-приложения «Vitera».

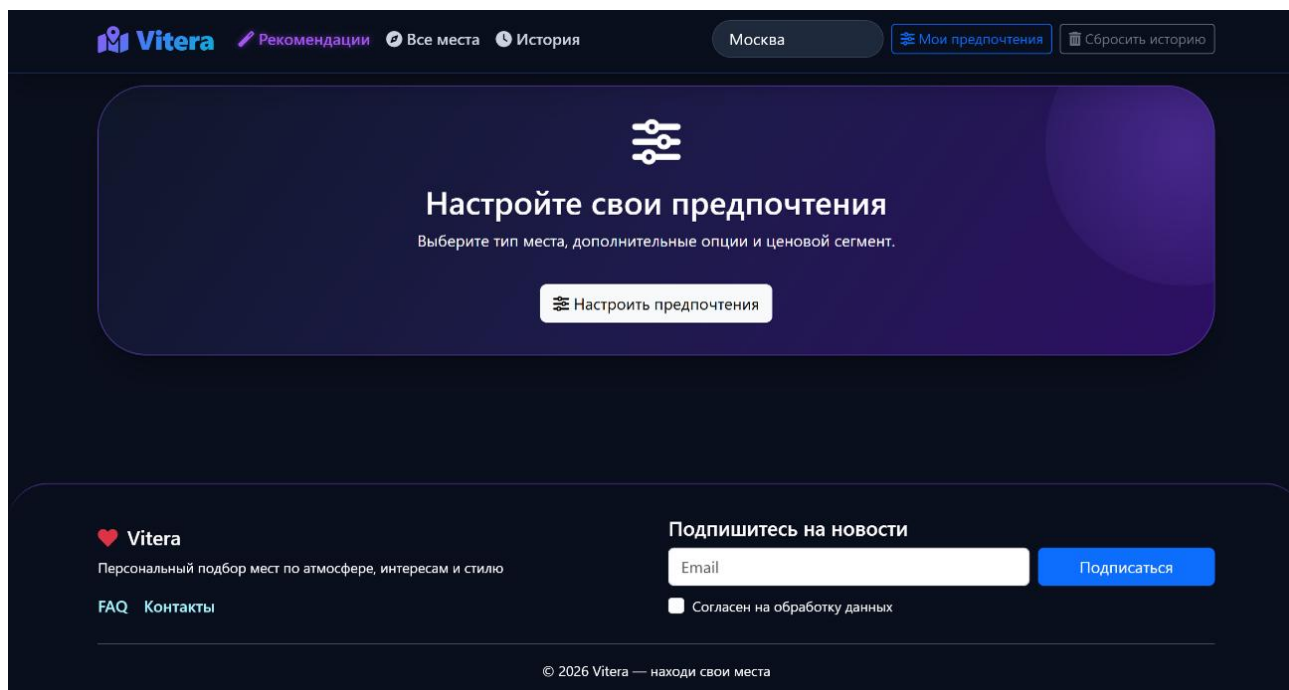


Рисунок 3.1 – Главная страница при первом запуске

На главной странице пользователю предлагается настроить предпочтения. Также доступна навигация по вкладкам «Рекомендации», «Все места», «История», выбор города и кнопки управления историей.

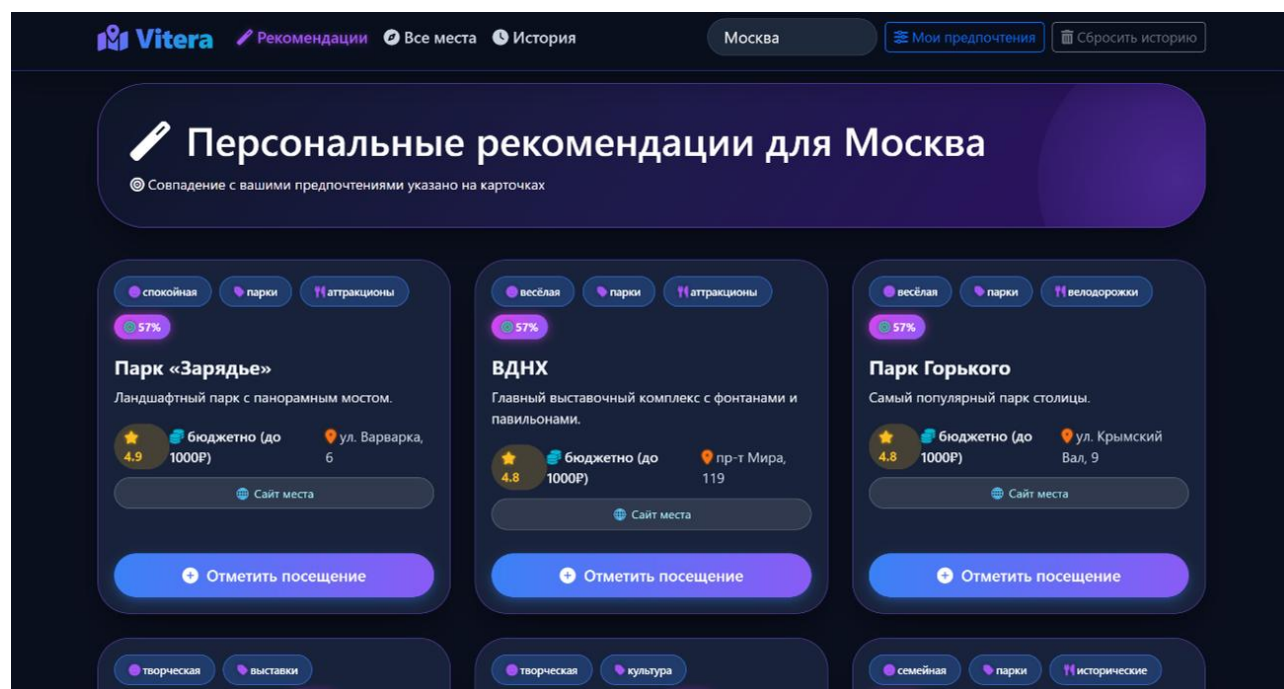
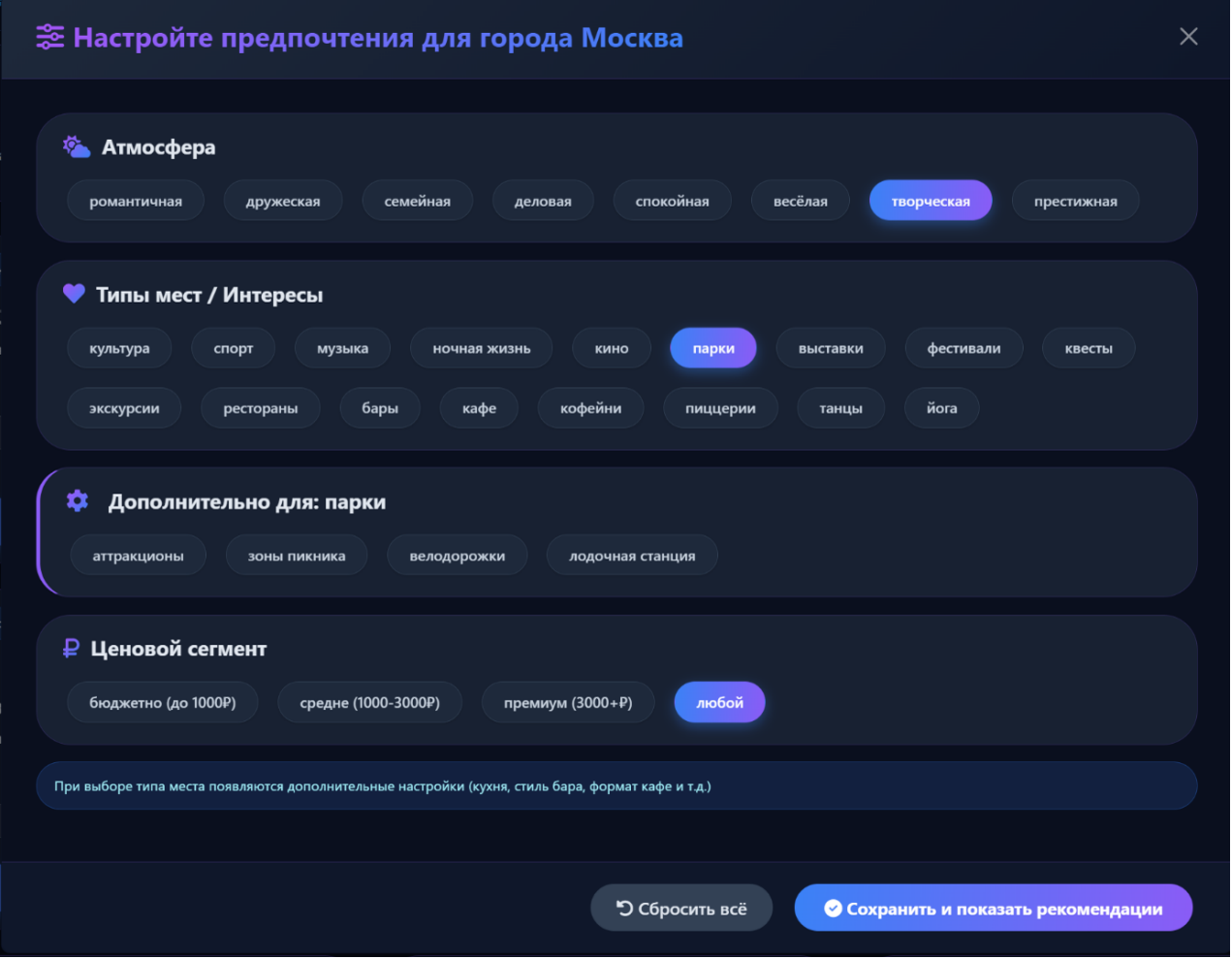


Рисунок 3.2 – Персональные рекомендации

После настройки предпочтений отображаются карточки мест с процентом совпадения, рейтингом, ценой, адресом и кнопкой «Отметить

посещение». Рекомендации сортируются по степени соответствия и популярности.



The image shows a dark-themed modal window titled "Настройте предпочтения для города Москва" (Set preferences for the city of Moscow). It contains four main sections for configuring preferences:

- Атмосфера** (Atmosphere): A row of buttons including "романтическая", "дружеская", "семейная", "деловая", "спокойная", "весёлая", "творческая" (selected), and "престижная".
- Типы мест / Интересы** (Types of places / Interests): A grid of buttons including "культура", "спорт", "музыка", "ночная жизнь", "кино", "парки" (selected), "выставки", "фестивали", "квесты", "экскурсии", "рестораны", "бары", "кафе", "кофейни", "пиццерии", "танцы", and "йога".
- Дополнительно для: парки** (Additional for: parks): A row of buttons including "аттракционы", "зоны пикника", "велодорожки", and "лодочная станция".
- Ценовой сегмент** (Price segment): A row of buttons including "бюджетно (до 1000Р)", "средне (1000-3000Р)", "премиум (3000+Р)", and "любой" (selected).

Below the sections, a note states: "При выборе типа места появляются дополнительные настройки (кухня, стиль бара, формат кафе и т.д.)" (When selecting a place type, additional settings appear (kitchen, bar style, cafe format, etc.)).

At the bottom, there are two buttons: "Сбросить всё" (Reset all) and "Сохранить и показать рекомендации" (Save and show recommendations).

Рисунок 3.3 – Модальное окно настройки предпочтений

В модальном окне доступны: выбор атмосферы, типа места, дополнительные опции (появляются динамически при выборе типа) и ценовой сегмент.

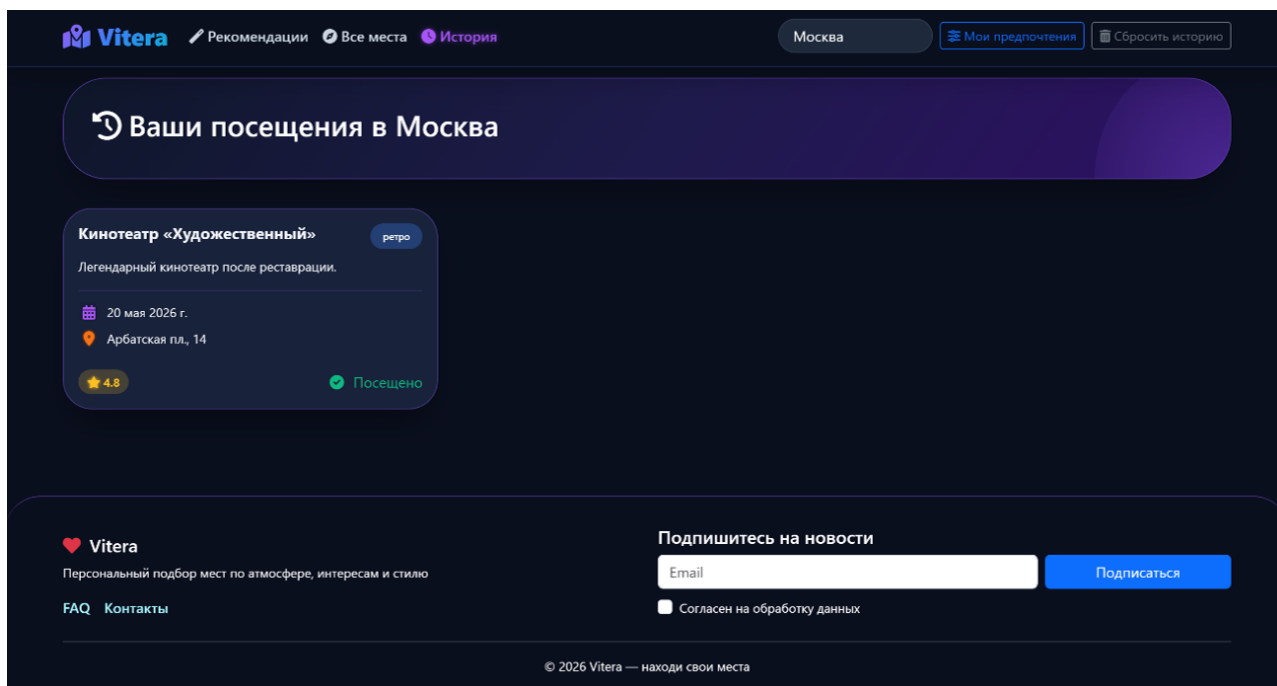


Рисунок 3.4 – Страница «История посещений»

В истории отображаются все отмеченные пользователем места с датой посещения, адресом и рейтингом. Предусмотрена возможность сброса истории.

3.4 Тестирование, оценка и рекомендации

В данном подразделе описывается процесс тестирования разработанного приложения и даются рекомендации по его дальнейшему развитию.

Функциональное тестирование было проведено вручную с использованием браузера Google Chrome и инструментов разработчика (консоль, вкладка Network). Проверялись следующие сценарии:

Таблица 3.1 – Результаты функционального тестирования

Сценарий	Ожидаемый результат	Фактический результат
Запуск приложения и загрузка данных	Интерфейс отображается, консоль выводит «Загружено 360 мест»	Успешно
Запрос GET /api/places	Возвращается JSON с 360+ объектами Place	Статус 200, данные корректны

Запрос GET /api/places/msk_1	Возвращается объект Кремля	Статус 200
Запрос с неверным id GET /api/places/xyz	Возвращается 404 с сообщением	Статус 404
Настройка предпочтений (выбор атмосферы, типа места, цены)	Появляются дополнительные опции, кнопки становятся активными	Успешно
Сохранение предпочтений	Закрывается модальное окно, появляются рекомендации с процентом совпадения	Процент совпадения рассчитывается корректно
Отметка места как посещённого	Кнопка меняет цвет и текст на «Посещено», место добавляется в историю	Успешно
Переход на вкладку «История»	Отображаются отмеченные места с датой посещения	Успешно
Смена города	Предпочтения и история для нового города загружаются, список мест обновляется	Успешно
Сброс истории	Все отмеченные места удаляются из истории, интерфейс обновляется	Успешно
Сброс предпочтений	Все выбранные настройки сбрасываются, модальное окно открывается заново	Успешно

Нагрузочное тестирование (неформальное): при одновременных запросах к API (например, через браузер) время ответа не превышало 50 мс, так как данные загружены в память.

Оценка:

- Приложение полностью соответствует функциональным требованиям.
- Интерфейс интуитивно понятен, не требует обучения.
- Бэкенд спроектирован с возможностью лёгкого расширения (подключение базы данных, добавление новых эндпоинтов).

Рекомендации по дальнейшему развитию:

- Добавление базы данных (PostgreSQL или SQLite) для хранения предпочтений и истории с возможностью синхронизации между устройствами.
- Реализация полнотекстового поиска по названиям и описаниям мест.
- Внедрение карт (Яндекс.Карты или Google Maps) для визуализации расположения мест.
- Разработка мобильного приложения на React Native или Flutter.
- Добавление системы аутентификации пользователей.
- Интеграция с реальными API для получения актуальных отзывов и рейтингов.

ЗАКЛЮЧЕНИЕ

Дальнейшее применение полученных результатов возможно в качестве самостоятельного веб-приложения, а также для интеграции с системами бронирования и лояльности.

В ходе выполнения работы было разработано веб-приложение «Vitera», которое полностью соответствует поставленным требованиям:

- Проведён анализ конкурентных решений, выявлены их недостатки (отсутствие персонализации и истории посещений).
- Определены проблемы целевой аудитории – сложность выбора места из-за отсутствия тонкой настройки предпочтений.
- Реализовано веб-приложение с функциями:
 - настройка предпочтений (атмосфера, тип места, субкатегория, ценовой сегмент);
 - персонализированные рекомендации с процентом совпадения;
 - просмотр всех мест выбранного города;
 - ведение истории посещений (localStorage);
 - смена города (12 крупнейших городов России).

В качестве технологического стека использованы современные решения: фронтенд на HTML/CSS/JS с Bootstrap 5, бэкенд на ASP.NET Core 10 (C#) с REST API. Приложение успешно протестировано и готово к использованию.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Задерновская М. С. Исследование электронной коммерции (увеличение доли интернет-продаж в 2020 году, факторы влияния) // Актуальные исследования. 2021. №34 (61). С. 40-46. URL: <https://apni.ru/article/2840-issledovanie-elektronnoj-kommertsii-uvelichen> (Дата обращения: 20.05.2026)
2. Маркетплейсы в России [Электронный ресурс] // TADVISER URL: <https://www.tadviser.ru/index.php/> Статья:Маркетплейсы_в_России (Дата обращения: 20.05.2026)
3. Сводные аналитические данные [Электронный ресурс] // АКИТ URL: <https://akit.ru/analytics/analyt-data> (Дата обращения: 20.05.2026)
4. Документация ASP.NET Core [Электронный ресурс] // Microsoft Learn URL: <https://learn.microsoft.com/ru-ru/aspnet/core/> (Дата обращения: 20.05.2026)
5. Документация Bootstrap 5 [Электронный ресурс] // Bootstrap URL: <https://getbootstrap.com/> (Дата обращения: 20.05.2026)